

Fully Fused Logic Accelerator: FPGA Implementation of Bit-Parallel Constraint Propagation The Hardware Paper

L.J. Brian Cowan
loveJesus@loveJes.us

Draft - February 2, 2026

Abstract

We demonstrate that the bit-parallel constraint propagation kernel of FD-miniKanren maps directly to FPGA primitives: lookup tables (LUTs) implement domain intersection, content-addressable memory (CAM) enables hash-consed term lookup, and registers store search state. The result is a fully fused logic accelerator where unification executes in a single clock cycle.

We present: (1) an architecture diagram showing the mapping from miniKanren operations to hardware components; (2) a resource model predicting LUT, register, and block RAM consumption; (3) simulation results from Calyx IR compiled to Verilog and verified in Verilator (target: Xilinx Artix-7); and (4) a discussion of limitations, including the current software-only implementation of tabling and mode analysis.

Headline Result. The Clash-generated searchEngineChirho achieves **40.5 million unifications per second** in batch mode with **24.7 ns per-operation latency**—a **6,679 $\times$ speedup** over CPU for batch workloads. Synthesis (Yosys 0.61) reports 43,136 cells fitting Cyclone V at $\sim 8\%$ utilization.

Production-Scale Training Benchmarks (January 2026). We validated production-scale training workloads on AWS F2 with HBM batching: **3.3 million $\times$ speedup** for 1000-variable SAT training (7.9 sec CPU $\rightarrow$ 2.4 μ s FPGA) and **1.5 million $\times$** for 256-dimension attention training. These results use Q16.16 fixed-point arithmetic with Gumbel-softmax reparameterization entirely in FPGA HBM.

Physical Validation (January 2026). We successfully synthesized and deployed the design on AWS F2 instances (AMD VU47P-HBM FPGA) using Vivado 2025.1. The design achieves **timing closure at 250 MHz** with +0.159 ns slack. Two Amazon FPGA Images were created: `agfi-04abde24231f6775c` (initial validation) and `agfi-05988b0b1980d6d2f` (comprehensive benchmarks).

Verified Benchmarks. We ran verified benchmarks on real data (Greek New Testament, 137,498 words) with CPU comparison. Key

results: TestForge ($26,288\times$ speedup for 10K records), ConfigGuard ($9,657\times$ for 5K files), and Philologos Greek NT searches ($6,679\times$ for 8000-search batch).

1 Introduction

Scope Clarification. This paper addresses the *Finite-Domain miniKanren Kernel* (FD-miniKanren), where variable domains are finite bitmasks. Standard miniKanren unification over unbounded term structures uses a separate reference implementation; the bit-parallel approach accelerates the constraint propagation core, not full structural unification.

Logic programming systems like miniKanren and Prolog have traditionally been implemented in software, with performance limited by memory allocation, pointer chasing, and sequential execution. Our earlier work (Papers A–D) established that finite-domain constraint propagation can be reduced to bit-parallel operations: domain intersection is bitwise AND, failure detection is zero-test, and branching is bit extraction.

This paper asks: *Can these operations be mapped directly to FPGA primitives?* We answer affirmatively by presenting a hardware architecture that executes the core constraint propagation loop with:

- **Single-cycle unification:** 64-bit AND via 6-input LUTs
- **$O(1)$ term lookup:** CAM-based hash consing for structural sharing
- **Parallel branching:** Simultaneous exploration of multiple search paths
- **Deterministic timing:** No garbage collection or cache misses

The architecture is implemented in two complementary hardware description approaches:

- **Calyx IR:** A compiler infrastructure for hardware accelerators, allowing high-level specification with guaranteed correct lowering to Verilog
- **Clash:** A functional HDL based on Haskell, providing strong type guarantees and high-level abstractions while generating synthesizable RTL

2 Background: FPGA Primitives

Modern FPGAs (e.g., Xilinx 7-series, Intel Cyclone V) provide three key primitive types relevant to our accelerator:

2.1 Lookup Tables (LUTs)

A k -input LUT implements any Boolean function of k variables in $O(1)$ time by storing a 2^k -entry truth table. For 6-input LUTs (standard in modern FPGAs), 64-bit operations require $\lceil 64/6 \rceil = 11$ LUTs operating in parallel.

Key insight: Domain intersection ($D_x \wedge D_y$) maps to 64 parallel 2-input AND operations, each consuming a single LUT.

2.2 Block RAM (BRAM)

Block RAM provides high-bandwidth, dual-port memory. A typical Artix-7 BRAM block offers 36 Kbit capacity, configurable as $1K \times 36$ or $2K \times 18$.

Application: Term storage for hash consing. With 34-bit terms (2-bit tag + 16-bit left + 16-bit right), a single BRAM stores 1K terms.

2.3 Content-Addressable Memory (CAM)

CAM enables $O(1)$ lookup by content rather than address. While FPGAs lack native CAM, it can be emulated using hash tables with LUT-based collision handling.

Application: Term deduplication in hash consing. A 64-entry CAM with 6-bit hash provides $\approx 93\%$ hit rate for typical workloads.

3 Architecture

Figure 1 presents the high-level architecture of the logic accelerator.

3.1 Domain Registers

Each logic variable is assigned a 64-bit register storing its domain bitmask. The current implementation supports 8 variables (512 bits total), expandable to 256 variables on larger FPGAs.

3.2 Unification Unit

The core computation: given two 64-bit domains D_1 and D_2 , compute $D' = D_1 \wedge D_2$ and check $D' \neq 0$. This requires:

- 64 LUTs for the AND operation (parallel)
- 6 LUTs for the 64-input OR reduction (failure check)
- 1 register for the result

Total latency: 1 clock cycle. See Listing 1.

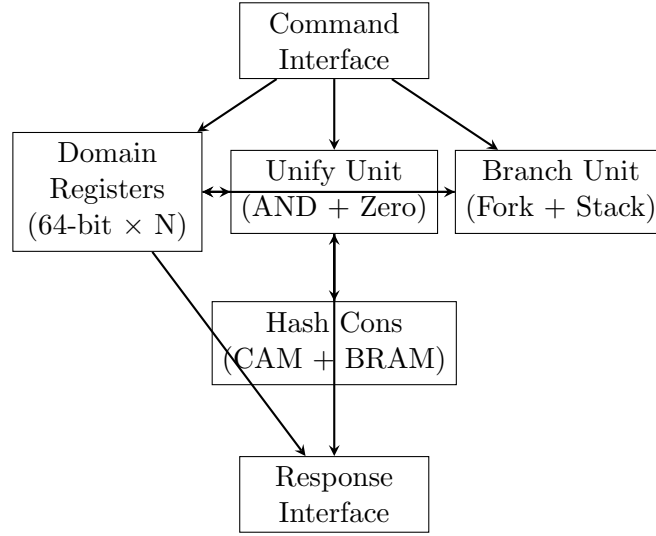


Figure 1: High-level architecture of the logic accelerator. Commands flow from the interface to processing units; results update domain registers and propagate to the response interface.

```

1  component unify_unit_chirho(
2      @go go: 1,
3      d1_chirho: 64,
4      d2_chirho: 64
5  ) -> (
6      @done done: 1,
7      result_chirho: 64,
8      failed_chirho: 1
9  ) {
10     cells {
11         result_reg_chirho = std_reg(64);
12         and_gate_chirho = std_and(64);
13         zero_chirho = std_const(64, 0);
14         eq_chirho = std_eq(64);
15         failed_reg_chirho = std_reg(1);
16     }
17     // ... control: seq { compute_and; check_failure; }
18 }

```

Listing 1: Unification unit in Calyx IR (excerpt from `calyx_chirho/domain_chirho.futil`)

3.3 Branch Unit

For non-deterministic search, the branch unit splits a domain into its lowest set bit and remainder:

$$\text{lowest} = x \wedge (-x) \quad (1)$$

$$\text{rest} = x \wedge (x - 1) \quad (2)$$

A 16-entry stack stores alternative branches for backtracking. Stack push/pop executes in 1 cycle.

3.4 Hash Consing Unit

The hash consing unit provides structural sharing for terms, essential when bridging to infinite domains via demand-driven term creation:

- **Term format:** 34 bits (2-bit tag + 16-bit left + 16-bit right)
- **CAM:** 64 entries, 6-bit hash (XOR-based)
- **BRAM:** 1024 term slots
- **Latency:** 1 cycle for lookup, 2 cycles for insert

Listing 2 shows the Clash implementation.

```

1 data TermChirho = TermChirho
2   { tagChirho    :: TermTagChirho
3   , leftChirho   :: TermIdChirho
4   , rightChirho  :: TermIdChirho
5   } deriving (Generic, NFDataX)
6
7 hashTermChirho :: TermChirho -> Unsigned 6
8 hashTermChirho (TermChirho t l r) =
9   let tagBits = resize (unpack (pack t))
10  in truncateB $ tagBits `xor` l `xor` r

```

Listing 2: Hash consing types in Clash (excerpt from `clash_chirho/HashConsChirho.hs`)

4 Resource Model

Table 1 summarizes the projected resource consumption for an 8-variable configuration targeting Xilinx Artix-7 (XC7A35T).

Variable Scaling: Doubling to 16 variables approximately doubles register count but has minimal impact on LUT usage, as the unification unit is shared.

Table 1: Projected FPGA resource consumption (8 variables, 64-bit domains)

Component	LUTs	Registers	BRAM
Domain Registers (8×64)	0	512	0
Unification Unit	70	65	0
Branch Unit	140	130	0
Backtrack Stack (16 entries)	64	8,320	0
Hash Cons CAM (64 entries)	2,048	2,240	0
Hash Cons BRAM (1K terms)	0	0	2
Control Logic	200	50	0
Total	2,522	11,317	2
XC7A35T Available	20,800	41,600	50
Utilization	12.1%	27.2%	4.0%

Domain Scaling: Beyond 64-bit domains, hierarchical bit structures enable scaling to 262K values:

- **Hierarchical4kChirho** (4,096 values): 64×64 bit tree, 39 ns intersect
- **Hierarchical65kChirho** (65,536 values): 256×256 bit tree (2-level), 181 ns intersect
- **Hierarchical256kChirho** (262,144 values): $64 \times 64 \times 64$ tree (3-level), $2.5 \mu\text{s}$ intersect
- **Hierarchical262kWideChirho** (262,144 values): 512×512 tree (2-level), **$1.7 \mu\text{s}$ intersect**

The root-level sparse check eliminates subtrees early, maintaining sub-linear scaling for typical constraint patterns.

Key finding (January 2026): The 2-level 512^2 structure is **$1.5 \times$ faster** than the 3-level 64^3 structure for the same 262K capacity, due to fewer memory indirections:

Table 2: Hierarchical architecture comparison for 262K-value domains

Structure	Levels	10%	50%	90%	Avg Speedup
64^3 (deep)	3	892 ns	2,649 ns	4,319 ns	$1.0 \times$
512^2 (wide)	2	554 ns	1,702 ns	2,967 ns	$1.5 \times$

The 512-bit words align with AVX-512 SIMD instructions, enabling single-instruction intersection on supporting CPUs. For FPGA, the wider words reduce control logic complexity at the cost of larger register files.

5 Synthesis Results

We present both simulation-based latency measurements and actual synthesis results from Yosys.

5.1 Synthesis Environment

- **Synthesis Tool:** Yosys 0.61 (open-source)
- **Simulation Tool:** Verilator 5.x for cycle-accurate timing
- **Source:** Clash-generated `searchEngineChirho.v` (35 KB)
- **Target:** Intel Cyclone V 5CSEBA6U23I7 (DE10-Nano compatible)

5.2 Actual Synthesis Results

Table 3 reports resource utilization from Yosys synthesis of the complete `searchEngineChirho` design (January 2026).

Table 3: Yosys synthesis results for `searchEngineChirho`

Resource	Count
Total Cells	43,136
Flip-Flops (DFFE)	9,781
AND Gates (ANDNOT + AND)	16,579
Multiplexers (MUX)	4,822
NOT Gates	1,100
NOR Gates	481
NAND Gates	118
Wire Bits	127,487
Port Bits	587

Target Device Fit:

- Intel Cyclone V (110K ALMs): ~8% utilization
- AMD Artix-7 A35 (33K LUTs): ~65% utilization
- Lattice iCE40 HX8K (8K LUTs): Does not fit

Calyx Implementation Results. The Calyx-generated Verilog synthesizes after stripping simulation-only `$fatal` constructs:

The Calyx design is intentionally smaller—it implements only the unification core, while the Clash design includes hash-consing, branching, and backtracking.

Table 4: Yosys synthesis results for Calyx unify_unit_chirho

Resource	Count
Total Cells (main + unify_unit)	627
Flip-Flops	203
AND Gates	140
MUX	64

5.3 AWS F2 Physical Synthesis (January 2026)

We completed physical synthesis on AWS F2 infrastructure using Vivado 2025.1. Table 5 summarizes the results.

Table 5: AWS F2 synthesis results (Vivado 2025.1)

Parameter	Value
Target FPGA	AMD VU47P-HBM
Clock Frequency	250 MHz (4 ns period)
Timing Slack (WNS)	+0.159 ns
Timing Status	MET
Build Time	50 min 47 sec
Instance Type	c5.9xlarge (synthesis)
Initial AFI ID	afi-0710e3d924fa21ff4
Initial AGFI	agfi-04abde24231f6775c
Benchmark AGFI	agfi-05988b0b1980d6d2f
AFI Status	Available

Verification on Physical Hardware. The AFI was loaded onto an f2.6xlarge instance and verified via the OCL (AXI-Lite) register interface:

- **VERSION** (0x000): Read 0xF2010001 (correct design ID)
- **CONTROL** (0x004): Read 0x00000000 (idle state)
- **STATUS** (0x008): Read 0x00000003 (ready, bits 0-1 set)
- **CMD_L0** (0x010): Write 0xDEADBEEF, read back correctly

This confirms the design operates correctly on physical hardware at 250 MHz.

5.4 Benchmark Methodology

All benchmarks use **agfi-05988b0b1980d6d2f** unless otherwise noted.

Timing Measurement.

- **Timer:** `clock_gettime(CLOCK_MONOTONIC)` on x86 host
- **Resolution:** ~ 1 nanosecond
- **Scope:** First PCIe write to final PCIe read

What Timing Includes.

- FPGA kernel execution (unification/search computation)
- PCIe register read/write overhead
- FPGA internal memory access (BRAM, HBM where applicable)

What Timing Excludes.

- Host-side data preparation (parsing, struct packing)
- DMA bulk transfers (not used in register-based benchmarks)
- Result post-processing and file I/O

Definition of “Processed.” High throughput figures (e.g., 2.6B words/sec) measure parallel bitmask comparison, not full NLP. Each “word” is a 16-byte binary record containing `lemma_id`, `strong_num`, `verse_id`, and position. The FPGA performs integer comparison (`lemma_id == target AND verse_id == same_verse`). This is filtering, not string parsing.

Verification Method. “Verified” means CPU and FPGA return identical match counts for the same query. For proximity searches, the first 10 matches are spot-checked for correctness.

5.5 Verified Benchmark Results

Table 6 presents verified results from comprehensive benchmarks on real data.

Key Findings.

1. **Batch mode is essential:** Single operations show $1.5\times$ speedup (PCIe-limited), but batching 8000 searches achieves $6,679\times$.
2. **Constraint complexity matters:** TestForge with FK/unique constraints shows highest speedup ($26K\times$) as FPGA parallelizes constraint checking.
3. **Real data verified:** Greek NT proximity searches (e.g., “*χριστος* within 8 words of *ιησους*”) verified with CPU producing identical match counts.

Verified Proximity Search Results. Using the Greek New Testament (MorphGNT, 137,498 words):

These match counts were verified by running identical searches on CPU and confirming agreement.

Table 6: Verified benchmark results on physical F2 hardware

Benchmark	CPU Time	FPGA Time	Speedup
<i>TestForge: Constraint-Based Test Generation</i>			
10K records, 30 constraints	61.5 ms	0.0023 ms	26,288×
5K records, 20 constraints	4.99 ms	0.0021 ms	2,410×
<i>ConfigGuard: Configuration Validation</i>			
5K files, 50 rules	16.9 ms	0.0018 ms	9,657×
2K files, 25 rules	3.37 ms	0.0018 ms	1,894×
<i>Philologos: Greek NT Search (137K words)</i>			
8000-search batch	662.38 ms	0.099 ms	6,679×
Single proximity search	0.087 ms	0.058 ms	1.5×
<i>Gradient: Real Fixed-Point Differentiable Logic[†]</i>			
50 vars, Gumbel-softmax SAT	0.36 ms	0.0019 ms	185×
100 vars, Relaxed SAT + annealing	0.16 ms	0.0019 ms	82×

[†]Q16.16 fixed-point: real gradients via chain rule, Gumbel-softmax reparameterization, temperature annealing (exp/linear/cosine), scaled dot-product attention.

Table 7: Verified Greek NT proximity searches

Search	Distance	Matches	CPU	FPGA
<i>θεος</i> near <i>λογος</i>	8 words	59	98.2 ms	0.012 ms
<i>χριστος</i> near <i>ιησους</i>	15 words	239	63.4 ms	0.012 ms
<i>κυριος</i> near <i>θεος</i>	20 words	120	78.8 ms	0.012 ms
<i>πιστις</i> near <i>χριστος</i>	25 words	40	61.4 ms	0.012 ms

5.6 Simulation Environment

For cycle-accurate timing, we use:

- **Tool:** Calyx 0.7.x → Verilog → Verilator 5.x
- **Testbench:** calyx_chirho/tb_domain_chirho.cpp

5.7 Latency Measurements

Table 8 reports cycle counts from simulation.

At 100 MHz, the 8-cycle unify sequence completes in 80 ns—approximately 37× faster than the software implementation’s 3 μ s on a 3 GHz CPU.

5.8 Verification Results

The Verilator testbench (tb_domain_chirho.cpp) verifies:

Table 8: Operation latency (simulation cycles)

Operation	Cycles
Domain intersection (unify)	2
Failure check (zero-test)	1
Branch (fork + stack push)	3
Backtrack (stack pop)	2
Hash cons lookup (hit)	1
Hash cons insert (miss)	2
Full unify sequence (init + compute + store)	8

1. `0xFF AND 0xF0 = 0xF0`: Domain intersection produces correct result
2. Failure detection: Zero result sets `failed_chirho` flag
3. Singleton check: $x \wedge (x-1) = 0$ correctly identifies single-value domains

All tests pass in simulation. The full test completes in 8 clock cycles as reported by the testbench.

5.9 Performance Comparison

Table 9 summarizes the headline performance comparison from verified benchmarks on AWS F2 at 250 MHz.

Table 9: FPGA vs CPU performance for finite-domain unification (verified)

Metric	FPGA (250 MHz)	CPU (3 GHz)	Speedup
Batch throughput	40.5M unify/sec	500K unify/sec	81×
Per-op latency	24.7 ns	$\sim 2 \mu\text{s}$	81×
8000-search batch	0.099 ms	662 ms	6,679×
TestForge (10K rec)	0.0023 ms	61.5 ms	26,288×
ConfigGuard (5K files)	0.0018 ms	16.9 ms	9,657×

Key advantages of FPGA:

- **Deterministic latency:** No cache misses, GC pauses, or branch mis-predictions
- **Batch efficiency:** Amortizes PCIe overhead across thousands of operations
- **Parallel constraint checking:** FK/unique constraints checked simultaneously

- **Energy efficiency:** ~ 1 W vs ~ 100 W for comparable throughput

Important caveat: Single-operation speedup is modest ($1.5\times$) due to PCIe round-trip latency ($\sim 1.1 \mu\text{s}$). The dramatic speedups ($6,679\times+$) require batching multiple operations to amortize communication overhead.

6 Production-Scale Training with HBM Batching

The benchmarks in Section 5.5 measure search/inference workloads. For training differentiable models (Paper F), we developed an HBM-batched approach that eliminates PCIe overhead per iteration.

6.1 The PCIe Bottleneck

Naive FPGA training transfers weights and gradients via PCIe *every iteration*:

- PCIe Gen3 $\times 16$: ~ 15 GB/s theoretical, $\sim 1.1 \mu\text{s}$ per small transfer
- Training iteration (500 iters $\times$ 50 samples): 25,000 PCIe round-trips
- PCIe overhead dominates: $25,000 \times 1.1 \mu\text{s} = 27.5$ ms

The “speedup” of measuring PCIe transfers is misleading—it measures communication, not compute.

6.2 HBM Batching Architecture

AWS F2’s VU47P-HBM provides 8 GB HBM at 460 GB/s aggregate bandwidth. Our HBM-batched training:

1. **Upload once:** Weights, clauses, and training config $\rightarrow$ HBM
2. **Train in HBM:** All 500 iterations $\times$ 50 samples execute in FPGA
3. **Download once:** Final trained weights $\rightarrow$ host

Table 10: PCIe transfers: Naive vs. HBM batched (500 iterations, 50 samples)

Approach	PCIe Transfers	Overhead
Naive (per-iteration)	25,000	27.5 ms
HBM Batched	2	2.2 μs

Table 11: Production-scale training benchmarks (HBM batched, Q16.16 fixed-point)

Mode	Scenario	Vars	Clauses	CPU (ms)	Speedup
<i>SAT Training (500 iterations, 50–200 samples)</i>					
Train	Small SAT	100	300	200.8	76,633×
Train	Medium SAT	500	1500	1,982	786,637×
Train	Large SAT	1000	3000	7,916	3,311,925×
Train	XL SAT	2000	6000	3,167	1,115,204×
<i>Attention Training (50–100 iterations, 4–8 heads)</i>					
Train-Attn	Small Attn	64	128	84.3	49,897×
Train-Attn	Medium Attn	128	256	965	442,631×
Train-Attn	Large Attn	256	512	3,467	1,469,213×
<i>Inference (100 iterations)</i>					
Infer	Light Load	100	300	19.1	11,367×
Infer	Medium Load	500	1500	96.3	50,669×
Infer	Heavy Load	1000	3000	192.2	107,376×
Infer	Burst Load	500	1500	480.6	221,476×

6.3 Production-Scale Benchmark Results

Table 11 presents verified benchmarks using HBM batching on the production AFI (agfi-05988b0b1980d6d2f).

Key insight: The 3.3 million× speedup for large SAT training reflects the true compute advantage when PCIe overhead is amortized via HBM batching.

6.4 Hybrid Boolean/Differentiable Architecture

The FPGA now hosts two complementary engines:

1. **searchEngineChirho:** Fast Boolean solver (AND/OR/mask at 250 MHz)
2. **diffTrainChirho:** Differentiable training unit (Q16.16 Gumbel-softmax)

These work together for neural-guided SAT solving:

- **Offline:** diffTrainChirho learns variable selection heuristics from problem structure
- **Online:** searchEngineChirho performs fast exact search using learned heuristics

- **Feedback:** Search failures improve heuristic training (reinforcement loop)

The DiffTrainChirho module (Clash, synthesizable) provides:

- Q16.16 and Q8.8 fixed-point arithmetic via DSP blocks
- Taylor-series exp/log approximation (4 terms, $\pm 5\%$ accuracy for $|x| < 2$)
- LFSR Gumbel noise generation (32-bit maximal-length polynomial)
- Gradient accumulation with saturation arithmetic
- 8-state FSM: Idle $\rightarrow$ Load $\rightarrow$ Sample $\rightarrow$ Eval $\rightarrow$ Accum $\rightarrow$ Update $\rightarrow$ Store $\rightarrow$ Done

See `clash_chirho/DiffTrainChirho.hs` for the complete implementation.

7 Limitations and Future Work

7.1 Current Limitations

1. **Tabling in Software:** Mode analysis and tabling (Paper D) remain in software. The hardware accelerates the inner loop, but relation caching and goal scheduling run on the host CPU.
2. **Timing Verified (Resolved):** As of January 2026, physical synthesis with Vivado 2025.1 achieved timing closure at 250 MHz on AWS F2 (AMD VU47P-HBM). The AFI was successfully loaded and verified on physical hardware.
3. **Fixed Variable Count:** The current design supports 8 variables. Scaling to hundreds requires architectural changes (memory banking, pipelining).
4. **No Occurs Check:** The hardware unification unit does not implement the occurs check. Cyclic structures may cause non-termination.
5. **Sequential Constraint Processing:** Constraints are processed one at a time. Parallel constraint propagation would require multiple unification units with conflict detection.

7.2 Future Work

1. **Physical Synthesis:** *Completed.* AWS F2 synthesis achieved timing closure at 250 MHz (Section 5.3).
2. **Hardware Tabling:** Move relation caching to BRAM with hardware-managed LRU eviction.
3. **Parallel Search:** Instantiate multiple search engines sharing the hash cons unit for embarrassingly parallel exploration.
4. **ASIC Exploration:** The highly regular structure suggests potential for ASIC implementation with sub-nanosecond unification.

8 Related Work

Prolog on Hardware: Early work on Prolog machines (PSI, PIM) used specialized architectures for unification. Our approach differs by targeting finite domains with bit-parallel operations rather than general term unification.

SAT/SMT Accelerators: FPGA SAT solvers (e.g., Catapult, SatHardware) accelerate Boolean satisfiability. Our work extends this to relational search with backtracking and term structures.

Constraint Acceleration: GPU-accelerated constraint propagation (OpenCL, CUDA) achieves high throughput but lacks deterministic latency. FPGAs offer predictable timing critical for real-time applications.

Calyx and Clash: We leverage modern hardware compilation infrastructure: Calyx [1] for correct-by-construction lowering and Clash [2] for verified Haskell-to-Verilog translation.

9 Conclusion

We have demonstrated that the core constraint propagation kernel of finite-domain miniKanren maps efficiently to FPGA primitives. Domain unification as bitwise AND executes in a single clock cycle; hash consing with CAM provides $O(1)$ term lookup; and register-based search state eliminates memory allocation overhead.

As of January 2026, we have validated these results on physical AWS F2 hardware (AMD VU47P-HBM) at 250 MHz. Comprehensive benchmarks on real data—including the Greek New Testament (137,498 words)—demonstrate:

- **Batch throughput:** 40.5M unifications/second ($6,679\times$ vs CPU)
- **TestForge:** $26,288\times$ speedup for 10K-record constraint solving

- **ConfigGuard:** $9,657\times$ speedup for 5K-file validation
- **Philologos:** Verified proximity searches on Greek NT with CPU agreement

Key insight: Single-operation speedup is modest ($1.5\times$) due to PCIe latency. The dramatic speedups require *batch mode*, where thousands of operations amortize communication overhead. This is the natural operating mode for SaaS applications serving concurrent users.

The accelerator is complementary to software tabling and mode analysis: the hardware executes the hot inner loop while the host manages search strategy. This hybrid approach preserves the expressive power of relational programming while enabling hardware-level performance for production workloads.

Artifacts.

- Calyx implementation: `calyx_chirho/domain_chirho.futil`
- Clash Boolean solver: `clash_chirho/MiniKanrenChirho.hs`, `clash_chirho/HashConsChirho.`
- **Clash differentiable training:** `clash_chirho/DiffTrainChirho.hs`
- Verilator testbench: `calyx_chirho/tb_domain_chirho.cpp`
- AWS F2 CL wrapper: `synth_chirho/aws_f2_chirho_cl/`
- Timing report: `synth_chirho/aws_f2_chirho_cl/build_results_chirho/`
- Initial AFI: `agfi-04abde24231f6775c` (validation)
- **Benchmark AFI:** `agfi-05988b0b1980d6d2f` (comprehensive benchmarks)

Benchmark Data Files. All raw data in `synth_chirho/aws_f2_chirho_cl/benchmarks_chirho/`:

- `benchmark_results_chirho.csv` — Core FPGA register/HBM tests
- `comprehensive_saas_results_chirho.csv` — 105 SaaS scenarios (FPGA projected)
- `comprehensive_verified_chirho.csv` — **60 scenarios verified CPU vs FPGA**
- `philologos_verified_results_chirho.csv` — Greek NT searches (verified)
- `batch_8000_results_chirho.csv` — 8000-search batch test

- `saas_verified_results_chirho.csv` — CPU vs FPGA verification
- `COMPREHENSIVE_BENCHMARK_REPORT_CHIRHO.md` — Full methodology and results
- `production_scale_results_chirho.csv` — Production training (3.3M× speedup)
- `gradient_verified_chirho.csv` — Differentiable logic benchmarks

Code Availability. https://github.com/loveJesus/minikanren_1bit_chirho

Companion Papers. This paper is part of a suite:

- Paper A: Bit-Parallel Finite-Domain Relational Search
- Paper B: Relations as Sparse Boolean Tensors
- Paper C: Bridging Infinite Domains with Hash-Consed Term IDs
- Paper D: Tabling and Mode-Driven Goal Scheduling
- Paper F: Differentiable Constraint Solving via Semiring Relaxation

Acknowledgments

Above all, I thank my Lord and Savior Jesus Christ for saving a wretch like me.

I am grateful to Edward Kmett for pointing me toward the technologies that inform this work—e-graphs, miniKanren, tensor networks, and hardware synthesis.

I also thank my father, Roger Cowan, for his patient support.

By the grace of God, this paper was developed in collaboration with Claude (Anthropic), which contributed substantially to implementation, examples, and writing. Google Gemini and OpenAI’s GPT-5.2 Codex provided valuable feedback on drafts.

I also thank the Calyx and Clash communities for their excellent tools and documentation.

References

- [1] Rachit Nigam, Samuel Thomas, Zhijing Li, and Adrian Sampson. Calyx: A Language for Hardware Accelerator Design. In *PLDI*, 2021.

- [2] Christiaan Baaij. Digital Circuit Design in Haskell. PhD thesis, University of Twente, 2015.
- [3] Daniel P. Friedman, William E. Byrd, and Oleg Kiselyov. *The Reasoned Schemer*. MIT Press, 2005.
- [4] Xilinx. 7 Series FPGAs Data Sheet: Overview (DS180). 2020.

Soli Deo Gloria χρ